This document applies to the API description for the "single-channel image + IMU synchronization" function, i.e., using only one global shutter camera to acquire image and IMU data. Both the image and IMU have timestamps and can be synchronized.

This document is a supplementary description to the "ELP Multi-Channel Global Synchronization - WinSDK" and "ELP Multi-Channel Global Synchronization - LinuxSDK". For SDK development methods, please refer to the above two documents.Windows and Linux share the same API.

Structure Description

Image Format Enumeration

```
typedef enum CamFormat {
  FORMAT_UNKNOWN = 0,
  FORMAT_MJPG = 1,
  FORMAT_RGB24 = 2,
  FORMAT_YUV422 = 3,
  FORMAT_NV12 = 4
}CamFormat;
```

Device Information

```
typedef struct DeviceInfo {
  char name[256];
  char vidpid[16];
  uint32_t number;
}DeviceInfo;
```

Image Format Information

```
typedef struct FormatInfo {
  CamFormat fmt;
  int width;
  int height;
  int fps;
}FormatInfo;
```

Parsed Image Information

```
typedef struct CamData {
  uint8_t* data;
```

```
    int width;
    int height;
    int bufSize;
    CamFormat format;
    uint64_t startExpouseTime;
    uint64_t endExpouseTime;
    sICM42688_XYZ_float imu_data[11];
    sAK09940_XYZ_int mtt_data[5];
}CamData;
```

IMU and Magnetometer Information

```
  typedef struct {
    uint64_t  uTime;
    float     fAccData_X;
    float     fAccData_Y;
    float     fAccData_Z;
    float     fGyroData_X;
    float     fGyroData_Y;
    float     fGyroData_Z;
}sICM42688_XYZ_float;

  typedef struct sAK09940_XYZ_int {
    uint64_t    uTime;
    int32_t     iX;
    int32_t     iY;
    int32_t     iZ;
    float      Temp;
    int32_t     iStatusBit;
}sAK09940_XYZ_int;
```

Interface Function Description

SCAM_Initialize

```
Function Prototype:  int SCAM_Initialize();
```

Function: Initialize SDK resources

Parameters: None

Return Value: Returns 1 on success, 0 on failure

Usage: This function must be called to initialize before calling other APIs

SCAM_Release

```
Function Prototype:  void SCAM_Release();
```

Function: Release SDK resources

Parameters: None

Return Value: None

Usage: Call this function when exiting the program

SCAM_GetVersion

```
Function Prototype:  const char* SCAM_GetVersion();
```

Function: Get SDK version information

Parameters: None

Return Value: Returns the version information string

Usage: None

SCAM_GetLastError

```
Function Prototype:  const int SCAM_GetLastError();
```

Function: Get error code

Parameters: None

Return Value: Returns 0 if there is no error; returns an integer error code if there is an error

Usage: None

SCAM_GetErrorText

Function Prototype: const char* SCAM_GetErrorText(int errorCode);

Function: Get detailed error information

Parameters: errorCode: integer error code

Return Value: Returns the string information corresponding to the error code

Usage: None

SCAM_EnumDevices

Function Prototype: int SCAM_EnumDevices(DeviceInfo* devices, int maxCount, int* count);

Function: Enumerate device information

Parameters: devices: DeviceInfo pointer, used to store the detected device information

maxCount: Maximum capacity of devices

count: Integer pointer, stores the actual number of detected devices

Return Value: Returns 1 if enumeration succeeds, 0 if enumeration fails

Usage: None

SCAM_OpenDevice

Function Prototype: int SCAM_OpenDevice(uint32_t deviceIndex, CamCallback callback, void* userData);

Function: Open the deviceParameters: deviceIndex: the logical index of the device to be opened

callback: Data callback function (replaces the original SetImageCallback and StartCapture)

userData: Custom parameter for the callback function

Return Value: Returns 1 on success, 0 on failure

Usage: None

SCAM_CloseDevice

Function Prototype: SCAM_CloseDevice(uint32_t deviceIndex);

Function: Set the format of the specified device

Parameters: deviceIndex: the logical index of the device to be closed

Return Value: Returns 1 on success, 0 on failure

Usage: None

SCAM_GetDeviceFormats

Function Prototype: int SCAM_GetDeviceFormats(uint32_t deviceIndex, FormatInfo* formats, int maxCount, int* count);

Function: Get the device format

Parameters: deviceIndex: device logical index

formats: Array for storing format information

maxCount: Maximum number of formats to store

count: Stores the number of formats retrieved

Return Value: Returns 1 on success, 0 on failure

Usage: None

SCAM_SetDeviceFormat

Function Prototype: int SCAM_SetDeviceFormat(uint32_t deviceIndex, int formatIndex);

Function: Set the device format

Parameters: deviceIndex: device logical index

formatIndex: Index of the format to be set

Return Value: Returns 1 on success, 0 on failure

Usage: None

SCAM_SetImageFormat

Function Prototype: int SCAM_SetImageFormat(uint32_t devIndex, CamFormat format);

Function: Set the output image format

Parameters: devIndex: device logical index

format: Specify the output format (FORMAT_NV12)

Return Value: Returns 1 on success, 0 on failure

Usage: None

SCAM_GetDeviceFormatIndex

Function Prototype: int SCAM_GetDeviceFormatIndex(uint32_t deviceIndex);

Function: Get the index of the current format

Parameters: deviceIndex: device logical index

Return Value: Returns the current format index on success, 0 on failure

Usage: None

SCAM_SaveToJPG

Function Prototype: bool SCAM_SaveToJPG(const uint8_t* buf, CamFormat fmt, int w, int h, const char* filename, int quality);

Function: Save a JPG image

Parameters: buf: image data buffer
Fmt: Image format

w: Image width

h: Image height

filename: Path of the file to be saved

quality: Compression quality, default 85

Return Value: Returns true on success, false on failure

Usage: Call this function when a photo needs to be taken

SCAM_GetFrameRate

Function Prototype: float SCAM_GetFrameRate(uint32_t deviceIndex);

Function: Get the frame rate of the output

imageParameters: deviceIndex: device index

Return Value: Frame rate value

Usage: None

Callback function pointer

Function Prototype: typedef void (*CamCallback)(const CamData* image, void* userData);

Function: User-defined callback function

Parameters: image: Pointer to a set of image data, which internally contains CamData

userData: Custom parameter passed in by the user when registering the callback

Return Value: None

Usage: None